Virtual Threads: A Deep Dive into Lightweight Concurrency

Introduction

Java 21 introduced Virtual Threads as a game-changing feature for concurrent programming. This article explores how virtual threads revolutionize the way we handle concurrency by breaking the traditional 1:1 mapping between application threads and OS threads.

Understanding I/O-Bound vs CPU-Bound Operations

Before diving into virtual threads, it's crucial to understand the two types of operations that dominate application workloads.

CPU-Bound Operations

CPU-bound operations are tasks where the CPU is the bottleneck. The thread actively uses the CPU for computations.

Examples:

- Mathematical calculations
- Data processing and transformations
- Encryption/decryption
- Image/video processing
- Sorting large arrays
- Complex algorithms

```
// CPU-bound operation
public long fibonacci(int n) {
    if (n <= 1) return n;
    return fibonacci(n - 1) + fibonacci(n - 2); // CPU constantly working
}
```

Characteristics:

- Thread continuously uses CPU cycles
- No waiting or blocking
- Performance limited by CPU speed and cores
- More threads than CPU cores doesn't help (causes contention)

I/O-Bound Operations

I/O-bound operations are tasks where the thread spends most time waiting for external resources.

Examples:

- Database queries
- HTTP requests to external APIs
- File read/write operations
- Network socket operations
- Waiting for user input
- Message queue operations

```
// I/O-bound operation
public String fetchUserData(String userId) {
    // Thread waits here for network response
    String response = httpClient.get("https://api.example.com/users/" + userId);
    return response; // Most time spent waiting, not computing
}
```

Characteristics:

- Thread spends majority of time waiting
- CPU sits idle during wait periods
- Performance limited by I/O speed (network, disk, etc.)
- Thread is blocked but not doing useful work
- **This is where traditional threads waste resources**

The Waiting Problem with Platform Threads

```
// Platform thread waiting for I/O
Thread platformThread = new Thread(() -> {
    String data = database.query("SELECT * FROM users"); // Blocked here
    // Thread occupies 1MB of stack memory
    // OS thread is blocked and unavailable
    // CPU does nothing but thread resources are held
});
```

During the I/O wait:

- 1MB of stack memory is allocated but unused
- OS thread is blocked and unavailable for other work
- CPU could be doing something useful but isn't
- **Massive waste of resources when scaled to thousands of requests**

The Problem with Platform Threads

Memory Constraints

When you create a platform thread in Java, it allocates a significant amount of stack memory (typically 1MB by default). This memory is reserved upfront, whether the thread uses it or not.

```
// Each platform thread consumes ~1MB of stack memory
Thread platformThread = new Thread(() -> {
    // Your task
});
platformThread.start();
```

Why you can't create too many platform threads:

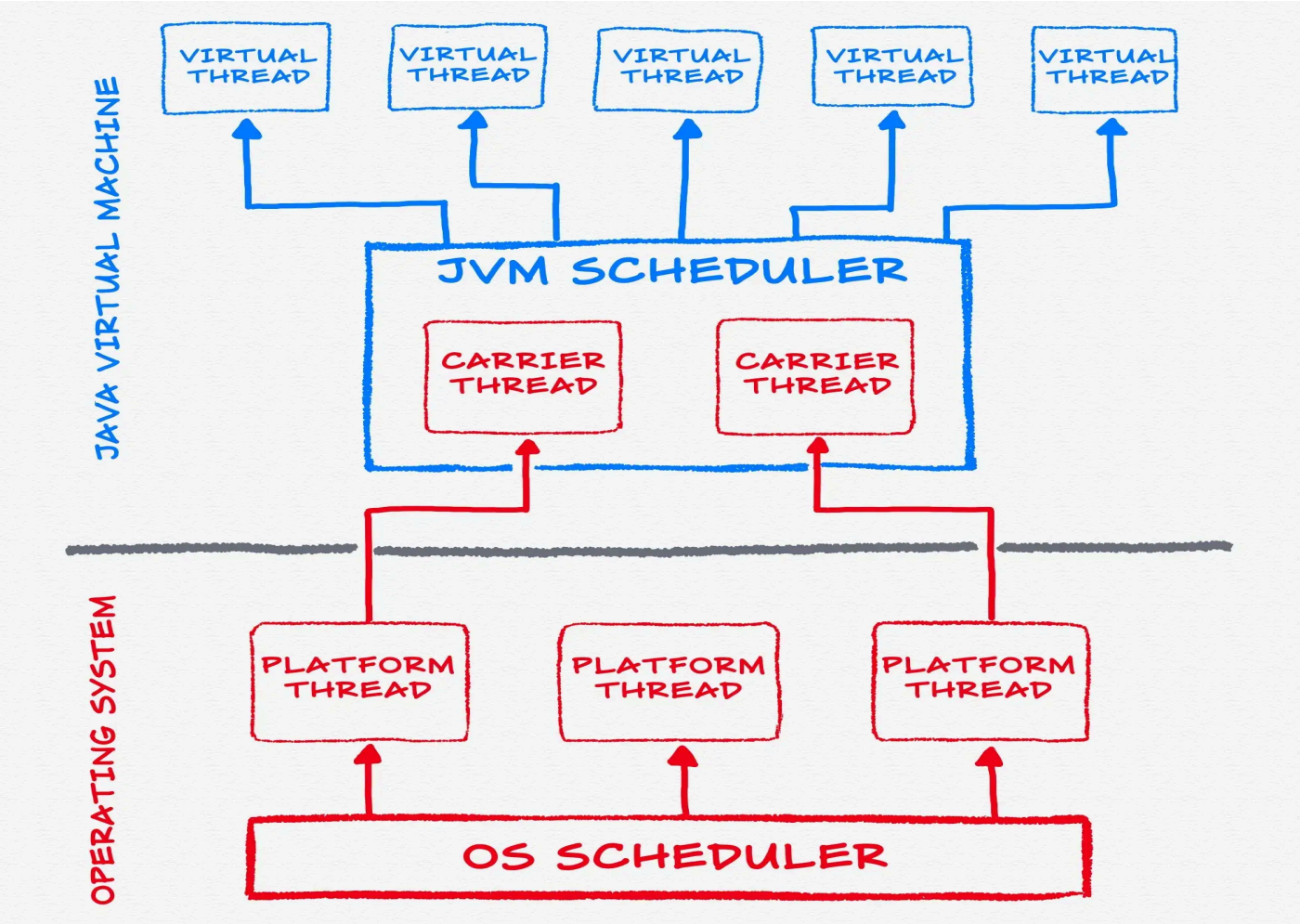
- Each thread requires dedicated stack memory (1MB default)
- Creating 10,000 threads = ~10GB of memory just for stacks
- This creates a hard limit on scalability

The 1:1 Mapping Problem

Platform threads have a 1:1 relationship with OS threads. This means:

- Each Java platform thread maps to exactly one OS thread
- OS threads are expensive resources managed by the kernel
- Context switching between OS threads involves kernel-level operations
- Limited by OS thread limits (typically thousands, not millions)

How Virtual Threads Work



The Carrier Thread Model

Virtual threads are managed by the JVM and run on a pool of platform threads called **carrier threads.** The JVM uses a ForkJoinPool as the default scheduler with one carrier thread per CPU core.

```
// Creating a virtual thread is cheap
Thread virtualThread = Thread.startVirtualThread(() -> {
    // Your task
});
```

Mount and Unmount Mechanism

This is where the magic happens:

Mounting:

- When a virtual thread needs to execute, the JVM scheduler assigns it to an available carrier thread (platform thread)
- The virtual thread's execution state is loaded onto the carrier thread's stack
- The virtual thread now runs on that carrier thread

Unmounting:

- When a virtual thread performs a blocking operation (I/O, sleep, lock acquisition), it unmounts from its carrier thread
- The JVM takes a **snapshot of the virtual thread's stack**
- This snapshot is stored in the heap memory
- The carrier thread becomes free to run another virtual thread

Stack Snapshot Magic

When a virtual thread blocks:

1. **Stack Capture:** The JVM captures the current call stack of the virtual thread
2. **Heap Storage:** This stack snapshot is stored as a regular Java object in the heap
3. **Carrier Release:** The carrier thread is freed immediately
4. **State Preservation:** All local variables, method frames, and execution context are preserved

```
Thread.startVirtualThread(() -> {
    System.out.println("Running on: " + Thread.currentThread());
    Thread.sleep(1000); // Unmounts here, stack saved to heap
    System.out.println("Running on: " + Thread.currentThread());
    // Might be a different carrier thread
});
```

What Happens When All Platform Threads Are Busy?

This is a critical scenario:

1. **Virtual Thread Queuing:** If all carrier threads are busy executing virtual threads, new virtual threads that are ready to run are queued
2. **FIFO Scheduling:** The scheduler maintains a queue of runnable virtual threads
3. **No Blocking:** Unlike platform threads, virtual threads don't block waiting for a carrier thread
4. **Automatic Mounting:** As soon as a carrier thread becomes available (when its current virtual thread unmounts), the scheduler assigns the next virtual thread from the queue

Important: This only becomes an issue if virtual threads are doing CPU-intensive work without blocking. For I/O-bound operations, virtual threads naturally unmount and free up carrier threads.

Memory Storage of Virtual Threads

Virtual threads are stored in different memory locations depending on their state:

Heap Memory

- **Stack snapshots:** When unmounted, the virtual thread's stack is stored as a Java object in the heap
- **Thread object metadata:** The Thread object itself and its associated data
- **Continuation objects:** Internal representation of the virtual thread's execution state

Platform Thread Stack (Temporarily)

- **During execution:** When mounted, the virtual thread's active stack frames use the carrier thread's stack
- **Shallow stacks:** Virtual threads typically have much smaller stacks than platform threads

```
Heap Memory (Virtual Thread Components):
├── Thread Object
├── Stack Snapshot (when unmounted)
├── Continuation Object
└── Thread-local variables
```

```
Carrier Thread Stack:
└── Active execution frames (when mounted)
```

What Happens When Virtual Thread Job Completes?

When a virtual thread finishes execution:

1. **Unmounting:** The virtual thread unmounts from its carrier thread
2. **Cleanup:** Thread-local variables are cleared
3. **Garbage Collection:** The virtual thread object becomes eligible for GC
4. **No Thread Pooling:** Unlike platform threads, virtual threads are not pooled or reused
5. **Carrier Thread Release:** The carrier thread becomes available for other virtual threads

```
Thread.startVirtualThread(() -> {
    performTask();
    // After this, virtual thread is eligible for GC
    // Carrier thread is freed immediately
});
```

Why Creating Virtual Threads Is Cheap

Multiple factors make virtual thread creation lightweight:

1. No OS Thread Creation

- Virtual threads don't create OS threads
- No kernel-level system calls required
- No context switching to kernel mode

2. Minimal Memory Footprint

- Initial stack size can be as small as a few kilobytes
- Stack grows dynamically as needed
- Stack is stored in heap, which is managed efficiently by GC

3. Lazy Allocation

- Memory is allocated on-demand
- No upfront reservation of large stack space

4. JVM-Level Management

- Creation and scheduling happen entirely in user space
- No interaction with OS scheduler for virtual thread management

```
// This is perfectly fine and cheap
for (int i = 0; i < 1_000_000; i++) {
    Thread.startVirtualThread(() -> {
        performLightweightTask();
    });
}
```

Creating Millions of Virtual Threads

Here's how the JVM manages millions of virtual threads on a handful of platform threads:

The Architecture

```
1,000,000 Virtual Threads
        ↓
    Scheduler
```

```
        ↓
8 Carrier Threads (Platform Threads)
        ↓
8 OS Threads
```

The Process

Creation Phase:

- 1 million virtual thread objects created in heap (~small memory footprint each)
- No OS threads created beyond the carrier thread pool

Execution Phase:

- Scheduler assigns virtual threads to available carrier threads
- When a virtual thread blocks (I/O, sleep), it unmounts
- Carrier thread immediately picks up next virtual thread

1. **Memory Usage:**

- 8 carrier threads × 1MB = 8MB for platform thread stacks
- 1 million virtual threads × ~1KB (avg) = ~1GB for virtual thread data
- Total: ~1GB vs ~1TB for 1 million platform threads

Example Code

```
public class VirtualThreadDemo {
    public static void main(String[] args) {
        try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
            // Submit 1 million tasks
            for (int i = 0; i < 1_000_000; i++) {
                int taskId = i;
                executor.submit(() -> {
                    // Simulating I/O operation
                    Thread.sleep(1000);
                    System.out.println("Task " + taskId + " on " +
                        Thread.currentThread());
                    return null;
                });
            }
        } // Executor waits for all tasks to complete
    }
}
```

Virtual Thread Switching Between Platform Threads

When a virtual thread is allocated to a different carrier thread after unmounting:

What Happens

1. **Stack Restoration:** The virtual thread's stack snapshot is loaded from heap
2. **Continuation Resume:** The execution continues from where it left off

3. **Thread-Local Preservation:** Thread-local variables remain intact
4. **No Data Loss:** All local variables and execution state are preserved

Example

```
Thread.startVirtualThread(() -> {
    long carrierThreadId1 = getCurrentCarrierThreadId();
    System.out.println("Before blocking: Carrier " + carrierThreadId1);

    Thread.sleep(100); // Unmounts here

    long carrierThreadId2 = getCurrentCarrierThreadId();
    System.out.println("After blocking: Carrier " + carrierThreadId2);
    // carrierThreadId1 might differ from carrierThreadId2

    // But all local variables and state are preserved!
});
```

Important Implications

- **No Thread Affinity:** Virtual threads don't have affinity to specific carrier threads
- **Transparent Migration:** The migration is completely transparent to the code
- **ThreadLocal Behavior:** ThreadLocal variables stay with the virtual thread, not the carrier

Virtual Threads: JVM-Level Abstraction

OS Thread Ignorance

The operating system has absolutely no knowledge of virtual threads:

```
Application Code
       ↓
Virtual Threads (JVM manages)
       ↓
Carrier Threads (JVM manages)
       ↓
Platform Threads (OS knows about these)
       ↓
OS Scheduler
       ↓
CPU Cores
```

Benefits of JVM-Level Management

1. **User-Space Scheduling:** No kernel involvement in virtual thread scheduling
2. **Fast Context Switching:** Switching virtual threads is just a method call
3. **Platform Independence:** Works the same across different operating systems
4. **JVM Optimizations:** JVM can optimize scheduling based on application behavior

Additional Important Concepts About Virtual Threads

1. Pinning

Virtual threads can become "pinned" to their carrier thread in certain situations:

```
synchronized (lock) {
    // Virtual thread is PINNED during synchronized block
    Thread.sleep(1000); // Won't unmount!
}
```

Causes of Pinning:

- Executing inside a `synchronized` block
- Executing native methods or foreign functions

Solution: Use `ReentrantLock` instead of `synchronized`:

```
ReentrantLock lock = new ReentrantLock();
lock.lock();
try {
    Thread.sleep(1000); // Virtual thread can unmount
} finally {
    lock.unlock();
}
```

2. Thread Pooling Anti-Pattern

Don't pool virtual threads! They're designed to be created per-task:

```
// BAD - Don't do this
ExecutorService pool = Executors.newFixedThreadPool(100); // Platform threads

// GOOD - Do this
ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor();
```

3. Structured Concurrency

Virtual threads work beautifully with structured concurrency:

```
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
    Future<String> user = scope.fork(() -> fetchUser());
    Future<String> order = scope.fork(() -> fetchOrder());

    scope.join();
    scope.throwIfFailed();

    return new Response(user.resultNow(), order.resultNow());
}
```

4. Observability

Virtual threads integrate with Java's monitoring tools:

```
// Thread dumps show virtual threads
jcmd <pid> Thread.print
```

```
// JFR events for virtual thread events
// JMX beans expose virtual thread metrics
```

5. Performance Characteristics

Virtual Threads Excel At:

- I/O-bound operations
- High-concurrency scenarios
- Blocking operations (sleep, network, file I/O)

Virtual Threads Struggle With:

- CPU-intensive tasks
- Code with heavy use of `synchronized`
- Extremely short-lived tasks (overhead of scheduling)

Why Virtual Threads Excel at I/O-Bound Operations

Virtual threads fundamentally solve the waiting problem that plagues platform threads in I/O-bound scenarios.

The Platform Thread Problem with I/O

Traditional web server with platform threads:

```
// Traditional approach with platform threads
ExecutorService threadPool = Executors.newFixedThreadPool(200);
```

```
// Handling HTTP request
threadPool.submit(() -> {
    // 1. Thread receives request (1ms of CPU work)
    Request request = receiveRequest();

    // 2. Query database (50ms waiting for I/O)
    Data data = database.query(request.getUserId()); // BLOCKED HERE

    // 3. Call external API (100ms waiting for network)
    String response = httpClient.get(apiUrl); // BLOCKED HERE

    // 4. Process and send response (1ms of CPU work)
    sendResponse(process(data, response));
});
```

Resource Waste Analysis:

- Total time: 152ms
- Actual CPU work: 2ms (1.3% of time)
- Waiting time: 150ms (98.7% of time)
- Thread holds 1MB stack memory for entire 152ms
- OS thread is blocked and unavailable
- **With 200 threads = 200MB of mostly idle memory**

How Virtual Threads Transform I/O-Bound Operations

Same operation with virtual threads:

```
// Virtual thread approach
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    executor.submit(() -> {
        // 1. Thread receives request (1ms) - MOUNTED
        Request request = receiveRequest();

        // 2. Query database (50ms) - UNMOUNTS HERE
        Data data = database.query(request.getUserId());
        // Virtual thread unmounts, carrier thread freed immediately!

        // 3. Call external API (100ms) - UNMOUNTS HERE
        String response = httpClient.get(apiUrl);
        // Again unmounts, carrier thread serves other requests

        // 4. Process response (1ms) - REMOUNTED
        sendResponse(process(data, response));
    });
}
```

Efficiency Gains:

- Carrier thread only occupied for 2ms of actual work
- During 150ms of waiting, carrier thread handles ~75 other requests
- Same 8 carrier threads can handle thousands of concurrent I/O operations
- Memory efficient: virtual thread stack snapshot only during waiting

Real-World I/O-Bound Example

Let's simulate a realistic scenario:

```
import java.net.http.HttpClient;
import java.time.Duration;
import java.util.concurrent.Executors;
```

```
public class VirtualThreadIODemo {

    public static void main(String[] args) throws Exception {
        int numberOfRequests = 10_000;
```

```
        System.out.println("Processing " + numberOfRequests + " I/O-bound requests");

        long start = System.currentTimeMillis();

        try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
            for (int i = 0; i < numberOfRequests; i++) {
                int requestId = i;
                executor.submit(() -> handleIOBoundRequest(requestId));
            }
        } // Waits for all tasks to complete

        long duration = System.currentTimeMillis() - start;
        System.out.println("Completed in: " + duration + "ms");
    }

    private static void handleIOBoundRequest(int requestId) {
        try {
            // Simulate database query (I/O operation)
            Thread.sleep(50); // Virtual thread unmounts here

            // Simulate external API call (I/O operation)
            Thread.sleep(100); // Virtual thread unmounts here

            // Simulate file write (I/O operation)
            Thread.sleep(30); // Virtual thread unmounts here

            // Minimal CPU work
            String result = "Request " + requestId + " processed";

        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

With Platform Threads (200 thread pool):

- Would need to process in batches
- 10,000 requests / 200 threads = 50 batches
- Each batch: 180ms (50ms + 100ms + 30ms waiting)
- Total time: ~9,000ms (9 seconds)
- Memory: 200MB for thread stacks

With Virtual Threads:

- All 10,000 requests start immediately
- Limited only by carrier threads (typically 8–16)
- Carrier threads constantly recycled during I/O waits
- Total time: ~200ms (limited by longest operation)
- Memory: ~10–20MB for virtual thread objects
- **45x faster and 10x less memory!**

The Virtual Thread I/O Workflow

Let's trace what happens step-by-step:

```java
Thread.startVirtualThread(() -> {
    // Step 1: MOUNTED on Carrier-1
    System.out.println("Starting request on: " + Thread.currentThread());

    // Step 2: I/O operation begins - UNMOUNTS
    String dbResult = database.query("SELECT * FROM users");
    // During this:
    // - Virtual thread stack snapshot saved to heap
    // - Carrier-1 freed to run VirtualThread-2
    // - When I/O completes, virtual thread queued for remounting

    // Step 3: I/O complete - REMOUNTED (maybe on Carrier-2)
    System.out.println("After DB query on: " + Thread.currentThread());

    // Step 4: Another I/O operation - UNMOUNTS again
    String apiResult = httpClient.get("https://api.example.com/data");
    // - Again unmounts, carrier freed
    // - Another virtual thread uses the carrier

    // Step 5: REMOUNTED (maybe on Carrier-3)
    System.out.println("After API call on: " + Thread.currentThread());

    // Step 6: Processing complete
    return processResults(dbResult, apiResult);
});
```

Key Points:

1. Virtual thread mounts only when doing actual work
2. During I/O waits, carrier thread is 100% available for others
3. Can switch carrier threads between I/O operations
4. No wasted resources during waiting periods

Throughput Comparison

Scenario: Web application handling requests with I/O latency

Platform Thread Pool (200 threads):

```
Throughput = Pool Size / Average Response Time
= 200 threads / 0.150 seconds
= ~1,333 requests/second
```

Virtual Threads (8 carrier threads):

```
Throughput = Carrier Threads / Active CPU Time
= 8 threads / 0.002 seconds
= ~4,000 requests/second
```

3x improvement with 25x less memory usage!

Why Virtual Threads Don't Help CPU-Bound Operations

```
// CPU-bound operation
Thread.startVirtualThread(() -> {
    // Computing fibonacci - CPU constantly busy
    for (int i = 0; i < 1000000; i++) {
        fibonacci(40); // No I/O, no unmounting possible
    }
    // Virtual thread STAYS MOUNTED on carrier thread
    // No benefit over platform threads
    // Actually slightly worse due to scheduling overhead
});
```

For CPU-bound tasks:

- Virtual thread never unmounts (no I/O to wait for)
- Occupies carrier thread continuously
- Limited by number of CPU cores
- Use platform threads or parallel streams instead

When to Use Virtual Threads vs Platform Threads

Use Virtual Threads for:

- REST API servers handling many concurrent requests
- Microservices with external service calls
- Database-heavy applications
- File processing pipelines
- WebSocket servers
- Any scenario with lots of waiting/blocking

Use Platform Threads for:

- CPU-intensive computations
- Parallel processing of data (use ForkJoinPool)
- Real-time systems requiring thread affinity
- Code heavily using `synchronized` blocks

The Sweet Spot: Mixed I/O Workloads

Virtual threads truly shine in real-world applications with mixed workloads:

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    // Handle 50,000 concurrent user requests
    for (int i = 0; i < 50_000; i++) {
        executor.submit(() -> {
            // Authenticate (DB call - I/O) - unmounts
            User user = authService.authenticate(token);

            // Fetch user data (DB call - I/O) - unmounts
```

```
            UserData data = userService.getData(user.getId());

            // Call recommendation service (API call - I/O) - unmounts
            List<Item> recommendations = recommendationService.get(user);

            // Call inventory service (API call - I/O) - unmounts
            List<Item> available = inventoryService.checkAvailability(recommendations);

            // Light processing (CPU)
            return formatResponse(data, available);
        });
    }
}
```

With platform threads, you'd need 50,000 threads (50GB memory) or batch processing. With virtual threads, 8–16 carrier threads handle everything efficiently!

6. Backward Compatibility

Virtual threads implement the same `Thread` API:

```
Thread vThread = Thread.startVirtualThread(() -> {
    // Use all standard Thread methods
    Thread.currentThread().getName();
    Thread.sleep(100);
    Thread.interrupted();
});
```

7. Carrier Thread Configuration

You can configure the carrier thread pool:

```
// Set number of carrier threads
System.setProperty("jdk.virtualThreadScheduler.parallelism", "16");

// Set carrier thread name prefix
System.setProperty("jdk.virtualThreadScheduler.namePrefix", "MyCarrier-");
```

8. Memory Pressure

Since virtual thread stacks live in heap:

- GC pressure can increase with millions of virtual threads
- Monitor heap usage and GC behavior
- Use appropriate heap size settings

9. Debugging Considerations

- Stack traces include carrier thread information
- Traditional thread dumps show both virtual and platform threads
- IDE debuggers are being updated for better virtual thread support

Best Practices

1. **Use virtual threads for I/O-bound tasks**
2. **Avoid `synchronized` blocks** - use `ReentrantLock`
3. **Don't pool virtual threads** — create them per-task
4. **Monitor pinning events** using JFR
5. **Design for blocking operations** — virtual threads shine here
6. **Test with realistic workloads** — measure actual improvement
7. **Keep carrier threads proportional to CPU cores**

Conclusion

Virtual threads represent a paradigm shift in Java concurrency. By decoupling application threads from OS threads and using clever mounting/unmounting mechanisms with heap-stored stack snapshots, Java 21 enables applications to handle millions of concurrent tasks with minimal resource overhead.

The key insights are:

- Virtual threads are cheap because they're JVM-managed and use minimal memory
- Mount/unmount with stack snapshots enables efficient carrier thread utilization
- Millions of virtual threads can run on a handful of platform threads
- The OS remains blissfully unaware of virtual threads
- Virtual threads naturally handle blocking operations without wasting resources

By embracing virtual threads, developers can write simple, blocking-style code that scales to millions of concurrent operations — the best of both worlds.